ASYNCHRÓNNE PROCESY

Čo je asynchrónne spracovanie?

Asynchrónne spracovanie znamená, že po spustení určitej operácie **nemusíme čakať na jej dokončenie** a môžeme pokračovať ďalšou operáciou, pričom výsledok spracujeme až vtedy, **keď bude dostupný**.

Asynchrónnosť sama osebe **neznamená vykonávanie JavaScriptu na viacerých vláknach**.

Call stack

Call stack je zásobník volaní funkcií, ktorý určuje, **čo sa práve vykonáva** a kam sa má kód po skončení funkcie vrátiť.

Nové volanie sa pridá **na vrch** a po dokončení sa odstráni. Kým sa volanie na vrchu nedokončí, funkcia pod ním nemôže pokračovať.

DEMO

Priebeh volaní

Spustenie príkladu. Spustíme runExample(). Prvý výpis je „Začiatok“.

Call stack

Vrch zásobníka je hore ↑

```
console.log('Začiatok')
```

```
runExample()
```

Console

Poradie výpisov

```
01 Začiatok
```

Synchrónne vykonávanie Zelená = práve vykonávané volanie. Ostatné volania čakajú pod ním.

Priebeh volaní

Objednávka. runExample() čaká, kým sa dokončí objednaj().

Call stack

Vrch zásobníka je hore ↑

```
console.log('Objednávka')
objednaj()
runExample()
```

Console

Poradie výpisov

```
01 Začiatok
02 Objednávka
```

Synchrónne vykonávanie Zelená = práve vykonávané volanie. Ostatné volania čakajú pod ním.

Priebeh volaní

Začiatok prípravy. objednaj() zavolá priprav(). Pred cyklom sa vypíše „Príprava“.

Call stack

Vrch zásobníka je hore ↑

```
console.log('Príprava')
priprav()
objednaj()
runExample()
```

Console

Poradie výpisov

```
01 Začiatok
02 Objednávka
03 Príprava
```

Synchrónne vykonávanie Zelená = práve vykonávané volanie. Ostatné volania čakajú pod ním.

Priebeh volaní

Dlhý cyklus. priprav() blokuje JS vlákno. „Odovzdanie“ musí počkať.

Call stack

Vrch zásobníka je hore ↑

```
priprav()
objednaj()
runExample()
```

Console

Poradie výpisov

```
01 Začiatok
02 Objednávka
03 Príprava
```

JS vlákno blokované 10 000 iterácií · približne 2 s alebo viac. Stále jedno volanie priprav().

Priebeh volaní

Príprava sa dokončila. priprav() sa odstráni zo zásobníka. Pokračuje objednaj().

Call stack

Vrch zásobníka je hore ↑

`objednaj()`

`runExample()`

Console

Poradie výpisov

```
01 Začiatok
02 Objednávka
03 Príprava
```

Synchrónne vykonávanie Zelená = práve vykonávané volanie. Ostatné volania čakajú pod ním.

Priebeh volaní

Odovzdanie. Po dokončení prípravy sa vypíše „Odovzdanie“.

Call stack

Vrch zásobníka je hore ↑

```
console.log('Odovzdanie')
objednaj()
runExample()
```

Console

Poradie výpisov

```
01 Začiatok
02 Objednávka
03 Príprava
04 Odovzdanie
```

Synchrónne vykonávanie Zelená = práve vykonávané volanie. Ostatné volania čakajú pod ním.

Priebeh volaní

Koniec príkladu. objednaj() sa dokončila. runExample() vypíše „Koniec“.

Call stack

Vrch zásobníka je hore ↑

```
console.log('Koniec')
```

```
runExample()
```

Console

Poradie výpisov

```
01 Začiatok
02 Objednávka
03 Príprava
04 Odovzdanie
05 Koniec
```

Synchrónne vykonávanie Zelená = práve vykonávané volanie. Ostatné volania čakajú pod ním.

Priebeh volaní

Zásobník je prázdny. Posledný výpis aj runExample() sa dokončili.

Call stack

Vrch zásobníka je hore ↑

Prázdny zásobník

Console

Poradie výpisov

```
01 Začiatok
02 Objednávka
03 Príprava
04 Odovzdanie
05 Koniec
```

Hotovo Všetky volania sú dokončené. Výpisy zostávajú v konzole.

DEMO

Task queue

Task queue je front pripravených úloh, ktoré čakajú na vykonanie. Ďalšia úloha sa môže spustiť, keď sa dokončí aktuálny JavaScriptový kód a **call stack sa vyprázdni** – nepreruší práve vykonávanú funkciu.

Callback je funkcia, ktorú odovzdáme inému kódu, aby ju zavolal.

Pri `setTimeout` sa čas meria **bez blokovania JS vlákna**. Callback sa do task queue nepridá hneď pri nastavení timeru, ale **až keď je pripravený na vykonanie**.

Task queue – príklady

V browserovom modeli sa ako tasks spracúvajú napríklad callbacky týchto API:

- `setTimeout(callback, delay)` – jednorazové spustenie po uplynutí oneskorenia.
- `setInterval(callback, delay)` – opakované spúšťanie v časových intervaloch.
- `addEventListener('click', callback)` – spracovanie kliknutia.
- `XMLHttpRequest.onload` – spracovanie dokončeného sieťového requestu.

React Native má vlastné prostredie, ale poskytuje aj `setTimeout` a `setInterval`.

DEMO

Priebeh volaní s timerom

Spustenie príkladu. Spustíme runExample(). Vypíše sa „Začiatok“.

Call stack

Vrch zásobníka je hore ↑

`console.log('Začiatok')`

`runExample()`

Prostredie / Timer

Mimo call stacku

Žiadny timer

Task queue

Pripravené callbacky

Prázdna

Console

Poradie výpisov

```
01 Začiatok
```

Synchrónne vykonávanie Zelená = práve vykonávané volanie. Vrch zásobníka je hore.

Priebeh volaní s timerom

Objednávka. runExample() zavolá objednaj(), ktorá vypíše „Objednávka“.

Call stack

Vrch zásobníka je hore ↑

```
console.log('Objednávka')
objednaj()
runExample()
```

Prostredie / Timer

Mimo call stacku

Žiadny timer

Task queue

Pripravené callbacky

Prázdna

Console

Poradie výpisov

```
01 Začiatok
02 Objednávka
```

Synchrónne vykonávanie Zelená = práve vykonávané volanie. Vrch zásobníka je hore.

Priebeh volaní s timerom

Príprava. objednaj() zavolá priprav(). Vypíše sa „Príprava“.

Call stack

Vrch zásobníka je hore ↑

```
console.log('Príprava')
priprav()
objednaj()
runExample()
```

Prostredie / Timer

Mimo call stacku

Žiadny timer

Task queue

Pripravené callbacky

Prázdna

Console

Poradie výpisov

```
01 Začiatok
02 Objednávka
03 Príprava
```

Synchrónne vykonávanie Zelená = práve vykonávané volanie. Vrch zásobníka je hore.

Priebeh volaní s timerom

Nastavenie timeru. Prostredie začne merať čas. Volanie setTimeout sa hneď vráti.

Call stack

Vrch zásobníka je hore ↑

`setTimeout(odovzdaj, cakanie)`

`priprav()`

`objednaj()`

`runExample()`

Prostredie / Timer

Mimo call stacku

`odovzdaj`

Čas sa meria mimo JS vlákna.

Task queue

Pripravené callbacky

Prázdna

Console

Poradie výpisov

01 Začiatok

02 Objednávka

03 Príprava

Timer čaká Callback ešte nie je v task queue. Synchrónny kód pokračuje.

Priebeh volaní s timerom

Pokračujeme ďalej. priprav() aj objednaj() sa dokončili. Bez čakania vypíšeme „Koniec“.

Call stack

Vrch zásobníka je hore ↑

```
console.log('Koniec')
```

```
runExample()
```

Prostredie / Timer

Mimo call stacku

`odovzdaj`

Čas sa meria mimo JS vlákna.

Task queue

Pripravené callbacky

Prázdna

Console

Poradie výpisov

```
01 Začiatok
02 Objednávka
03 Príprava
04 Koniec
```

Timer čaká Callback ešte nie je v task queue. Synchrónny kód pokračuje.

Priebeh volaní s timerom

Synchrónny kód sa dokončil. Timer ešte čaká, ale JS vlákno je voľné pre ďalšiu prácu.

Call stack

Vrch zásobníka je hore ↑

Prázdny zásobník

Prostredie / Timer

Mimo call stacku

`odovzdaj`

Čas sa meria mimo JS vlákna.

Task queue

Pripravené callbacky

Prázdna

Console

Poradie výpisov

01 Začiatok
02 Objednávka
03 Príprava
04 Koniec

JS vlákno je voľné Prázdny zásobník neznamená, že timer už uplynul.

Priebeh volaní s timerom

Uplynul čas timeru. Callback odovzdaj je pripravený a zaradí sa do task queue.

Call stack

Vrch zásobníka je hore ↑

Prázdny zásobník

Prostredie / Timer

Mimo call stacku

Čas uplynul

Callback je pripravený.

Task queue

Pripravené callbacky

```
function odovzdaj() {
  console.log('Odovzdanie');
}
```

Console

Poradie výpisov

```
01 Začiatok
02 Objednávka
03 Príprava
04 Koniec
```

Callback vo fronte Task queue obsahuje iba callbacky pripravené na vykonanie.

Priebeh volaní s timerom

Spustenie callbacku. Úloha sa dostala na rad: callback opustí front a spustí sa.

Call stack

Vrch zásobníka je hore ↑

```
function odovzdaj() {
  console.log('Odovzdanie');
}
```

Prostredie / Timer

Mimo call stacku

Timer dokončený

Task queue

Pripravené callbacky

Prázdna

Console

Poradie výpisov

```
01 Začiatok
02 Objednávka
03 Príprava
04 Koniec
```

Vykonáva sa callback priprav(), objednaj() ani runExample() sa už neobnovujú.

Priebeh volaní s timerom

Odovzdanie. Callback vypíše „Odovzdanie“ – až po výpise „Koniec“.

Call stack

Vrch zásobníka je hore ↑

`console.log('Odovzdanie')`

```
function odovzdaj() {
  console.log('Odovzdanie');
}
```

Prostredie / Timer

Mimo call stacku

Timer dokončený

Task queue

Pripravené callbacky

Prázdna

Console

Poradie výpisov

01 Začiatok
02 Objednávka
03 Príprava
04 Koniec
05 Odovzdanie

Vykonáva sa callback priprav(), objednaj() ani runExample() sa už neobnovujú.

Priebeh volaní s timerom

Callback sa dokončil. Výpis aj odovzdaj() sa dokončili. Zásobník aj front sú prázdne.

Call stack

Vrch zásobníka je hore ↑

Prázdny zásobník

Prostredie / Timer

Mimo call stacku

Timer dokončený

Task queue

Pripravené callbacky

Prázdna

Console

Poradie výpisov

```
01 Začiatok
02 Objednávka
03 Príprava
04 Koniec
05 Odovzdanie
```

Hotovo Poradie: Začiatok → Objednávka → Príprava → Koniec → Odovzdanie.

DEMO

Timer a plný call stack

Nastavíme timer na 500 ms. Prostredie začne merať čas. Callback zatiaľ nie je v task queue.

Call stack

Vrch zásobníka je hore ↑

```
setTimeout(poTimere, 500)
```

```
runExample()
```

Prostredie / Timer

Mimo call stacku

`poTimere · 500 ms`

Prostredie meria čas.

Task queue

Pripravené callbacky

Prázdna

Console

Ilustračný výstup

```
01 Začiatok
```

Timer čaká Začiatok príkladu · timer je nastavený na 500 ms.

Timer a plný call stack

Začneme blokujúci cyklus. setTimeout sa vrátil. blokuj() teraz vykonáva synchrónny while.

Call stack

Vrch zásobníka je hore ↑

`blokuj()`

`runExample()`

Prostredie / Timer

Mimo call stacku

`poTimere · 500 ms`

Prostredie meria čas.

Task queue

Pripravené callbacky

Prázdna

Console

Ilustračný výstup

```
01 Začiatok
```

JS vlákno blokované 500 ms ešte neuplynulo. Callback zatiaľ nie je pripravený.

Timer a plný call stack

Uplynulo 500 ms, cyklus stále beží. Callback je vo fronte, ale nemôže prerušiť bežiaci while.

Call stack

Vrch zásobníka je hore ↑

`blokuj()`

`runExample()`

Prostredie / Timer

Mimo call stacku

500 ms uplynulo

Callback je pripravený.

Task queue

Pripravené callbacky

```
function poTimere() {
  const uplynulo =
Math.round(performance.now() -
zaciatok);
  console.log(`Timer 500 ms:
spustený po ${uplynulo} ms`);
}
```

Console

Ilustračný výstup

```
01 Začiatok
```

JS vlákno blokované Timer je pripravený ≠ callback sa už vykonáva.

Timer a plný call stack

Cyklus skončil, vypíšeme Koniec. Približne po 2 sekundách sa blokuj() dokončí. runExample() pokračuje.

Call stack

Vrch zásobníka je hore ↑

```
console.log('Koniec')
```

```
runExample()
```

Prostredie / Timer

Mimo call stacku

500 ms uplynulo

Callback je pripravený.

Task queue

Pripravené callbacky

```
function poTimere() {
  const uplynulo =
Math.round(performance.now() -
zaciatok);
  console.log(`Timer 500 ms:
spustený po ${uplynulo} ms`);
}
```

Console

Ilustračný výstup

```
01 Začiatok
02 Koniec
```

Callback stále čaká Aj po cykle sa musí dokončiť zvyšok synchrónneho kódu.

Timer a plný call stack

Call stack sa vyprázdni. runExample() sa dokončila. Čakajúci callback sa môže dostať na rad.

Call stack

Vrch zásobníka je hore ↑

Prázdny zásobník

Prostredie / Timer

Mimo call stacku

500 ms uplynulo

Callback je pripravený.

Task queue

Pripravené callbacky

```
function poTimere() {
  const uplynulo =
Math.round(performance.now() -
zaciatok);
  console.log(`Timer 500 ms:
spustený po ${uplynulo} ms`);
}
```

Console

Ilustračný výstup

```
01 Začiatok
02 Koniec
```

JS vlákno je voľné Až teraz môže task queue odovzdať callback na vykonanie.

Timer a plný call stack

Vykonáme callback timeru. poTimere() vypíše skutočne uplynutý čas: približne 2 sekundy alebo viac.

Call stack

Vrch zásobníka je hore ↑

```
console.log(`Timer 500 ms: spustený po ${uplynulo} ms`)
```

```
function poTimere() {
  const uplynulo = Math.round(performance.now() - zaciatok);
  console.log(`Timer 500 ms: spustený po ${uplynulo} ms`);
}
```

Prostredie / Timer

Mimo call stacku

Timer dokončený

Čas vo výpise je ilustračný.

Task queue

Pripravené callbacky

Prázdna

Console

Ilustračný výstup

```
01 Začiatok
02 Koniec
03 Timer 500 ms: spustený po 2003 ms
```

Vykonáva sa callback

2003 ms je príklad výstupu, nie meranie tejto prezentácie.

Timer a plný call stack

Callback sa dokončil. Výpis aj poTimere() sa dokončili. Stack aj task queue sú prázdne.

Call stack

Vrch zásobníka je hore ↑

Prázdny zásobník

Prostredie / Timer

Mimo call stacku

Timer dokončený

Čas vo výpise je ilustračný.

Task queue

Pripravené callbacky

Prázdna

Console

Ilustračný výstup

```
01 Začiatok
02 Koniec
03 Timer 500 ms:
spustený po 2003 ms
```

Hotovo 500 ms je oneskorenie timeru, nie záruka presného času spustenia.

`Promise`

Objekt **Promise** predstavuje **budúce dokončenie alebo zlyhanie asynchrónnej operácie** a jej výslednú hodnotu.

Promise používame pri asynchrónnych operáciách, napríklad pri **HTTP požiadavkách**.

Vytvorenie `Promise`

```
const promise = new Promise((resolve, reject) => {
  // Tu spustíme operáciu a neskôr oznámime jej výsledok.
});
```

Funkcia **executor** sa vykoná **hneď pri vytvorení Promise**.

`resolve(value)` oznámi úspech a odovzdá výsledok.

`reject(reason)` oznámi zlyhanie a odovzdá dôvod.

`new Promise` vykoná executor synchrónne. Dlhý `while` v ňom stále blokuje JavaScript.

Stavy `Promise`

Stav	Význam	Výsledok
`pending`	Čaká na dokončenie	Zatiaľ nie je dostupný
`fulfilled`	Operácia bola úspešná	Hodnota
`rejected`	Operácia zlyhala	Dôvod odmietnutia

`fulfilled` a `rejected` sú konečné stavy: **settled**. Promise sa už nevráti do `pending`.

Konceptuálny model `Promise`

Pre vysvetlenie si Promise môžeme predstaviť takto:

```
stav: pending
výsledok: zatiaľ neznámy
reakcie na úspech: []
reakcie na chybu: []
```

Toto je **model interného mechanizmu**, nie vlastnosti dostupné cez `promise.stav` alebo `promise.výsledok`.

Spracovanie výsledku

Na Promise pripájame **handlery** – funkcie, ktoré sa vykonajú po jej dokončení.

Handler	Úloha	Argument
`.then(value => …)`	Spracuje úspešný výsledok.	`value` je hodnota odovzdaná do `resolve`.
`.catch(error => …)`	Spracuje chybu z predchádzajúcej časti reťazca.	`error` je dôvod odmietnutia.
`.finally(() => …)`	Spoločný dokončovací krok pri úspechu aj chybe.	Nedostáva hodnotu ani chybu ako argument.

Reťazec `then`, `catch` a `finally`

```
promise
  .then(value => {
    console.log('Úspech:', value);
  })
  .catch(error => {
    console.log('Chyba:', error.message);
  })
  .finally(() => {
    console.log('Spracovanie skončilo');
  });
```

Handlery sa vykonajú **až po aktuálnom synchrónnom kóde**, aj keď je Promise už splnená.

DEMO

Rozpísané reťazenie Promise

To isté môžeme rozpísať bez skráteného reťazca:

```
const prva = Promise.resolve(2);
const druha = prva.then(value => value + 3);
const tretia = druha.then(value => value * 4);

console.log(prva === druha); // false – ide o dva rôzne objekty

tretia.then(value => console.log(value)); // 20
```

Chýbajúci `return`

```
// Chyba: blokové telo bez return.
Promise.resolve(2)
  .then(value => {
    value + 3;
  })
  .then(value => console.log(value)); // undefined

// Oprava.
Promise.resolve(2)
  .then(value => {
    return value + 3;
  })
  .then(value => console.log(value)); // 5
```

DEMO

Microtask queue

Microtask queue je front callbackov, ktoré čakajú na vykonanie po dokončení aktuálneho synchrónneho kódu, ale **pred ďalšou task z task queue**.

Callbacky sa vykonávajú v poradí naplánovania. Event loop spracuje microtasks **až do vyprázdnenia frontu** a až potom môže spustiť ďalšiu pripravenú task.

Priorita neznamená prerušenie. Aj microtask musí počkať na prázdny call stack.

Čo sa plánuje do microtask queue?

Napríklad reakcie `.then()`, `.catch()`, `.finally()`, pokračovanie po `await` a funkcie odovzdané `queueMicrotask()`.

Executor v `new Promise(...)` sa vykoná **hneď**. Microtask je až naplánovaný handler.

Pri čakajúcej Promise sa handler naplánuje až vtedy, keď je známy príslušný výsledok.

Microtask môže vytvoriť ďalšiu microtask

Nová microtask sa pridá **na koniec microtask queue** a vykoná sa ešte **pred ďalšou task**.

Ak microtask stále vytvára ďalšiu microtask, front sa nemusí vyprázdniť. Ďalšie task zostanú čakať a **aplikácia môže prestať reagovať**.

DEMO

Objednávky: task a microtask

Prvá objednávka. Timer čaká; .then() naplánuje prvé potvrdenie. runExample() pokračuje.

Call stack

Vrch zásobníka je hore ↑

`objednaj(1)`

`runExample()`

Microtask queue

Začiatok frontu je hore ↑

Objednávka 1 · cislo = 1

```
function potvrd() {
  console.log(`Potvrdenie ${cislo}`);

  queueMicrotask(function
vystavDoklad() {
    console.log(`Doklad ${cislo}`);
  });
}
```

Task queue

Pripravené callbacky · začiatok hore ↑

Prázdna

Console

Poradie výpisov

```
01 Začiatok
02 Objednávka 1.
03 Príprava 1
```

Synchrónny kód Ani nulový timer nepreruší práve vykonávanú funkciu.

Objednávky: task a microtask

Dokončenie synchrónneho kódu. Druhá objednávka je vytvorená. Vypísal sa Koniec a runExample() sa dokončila.

Call stack

Vrch zásobníka je hore ↑

Prázdny zásobník

Microtask queue

Začiatok frontu je hore ↑

Objednávka 1 · cislo = 1

```
function potvrd() {
  console.log(`Potvrdenie ${cislo}`);

  queueMicrotask(function
vystavDoklad() {
    console.log(`Doklad ${cislo}`);
  });
}
```

Objednávka 2 · cislo = 2

```
function potvrd() {
  console.log(`Potvrdenie ${cislo}`);

  queueMicrotask(function
vystavDoklad() {
    console.log(`Doklad ${cislo}`);
  });
}
```

Task queue

Pripravené callbacky · začiatok hore ↑

Objednávka 1 · cislo = 1

```
function odovzdaj() {
  console.log(`Odovzdanie ${cislo}`);

  Promise.resolve().then(function
uzavri() {
    console.log(`Uzavretie ${cislo}`);
  });
}
```

Objednávka 2 · cislo = 2

```
function odovzdaj() {
  console.log(`Odovzdanie ${cislo}`);

  Promise.resolve().then(function
uzavri() {
    console.log(`Uzavretie ${cislo}`);
  });
}
```

Console

Poradie výpisov

```
01 Začiatok
02 Objednávka 1.
03 Príprava 1
04 Objednávka 2.
05 Príprava 2
06 Koniec
```

Najprv microtasks Od tohto kroku predpokladáme, že oba nulové timery sú už pripravené.

Objednávky: task a microtask

Prvé potvrdenie. potvrd() (cislo = 1) vypíše Potvrdenie 1 a pridá doklad na koniec microtask queue.

Call stack

Vrch zásobníka je hore ↑

Objednávka 1 · cislo = 1

```
function potvrd() {
  console.log(`Potvrdenie ${cislo}`);

  queueMicrotask(function
vystavDoklad() {
    console.log(`Doklad ${cislo}`);
  });
}
```

Microtask queue

Začiatok frontu je hore ↑

Objednávka 2 · cislo = 2

```
function potvrd() {
  console.log(`Potvrdenie ${cislo}`);

  queueMicrotask(function
vystavDoklad() {
    console.log(`Doklad ${cislo}`);
  });
}
```

Objednávka 1 · cislo = 1

```
function vystavDoklad() {
  console.log(`Doklad ${cislo}`);
}
```

Task queue

Pripravené callbacky · začiatok hore ↑

Objednávka 1 · cislo = 1

```
function odovzdaj() {
  console.log(`Odovzdanie ${cislo}`);

  Promise.resolve().then(function
uzavri() {
    console.log(`Uzavretie ${cislo}`);
  });
}
```

Objednávka 2 · cislo = 2

```
function odovzdaj() {
  console.log(`Odovzdanie ${cislo}`);

  Promise.resolve().then(function
uzavri() {
    console.log(`Uzavretie ${cislo}`);
  });
}
```

Console

Poradie výpisov

```
01 Začiatok
02 Objednávka 1.
03 Príprava 1
04 Objednávka 2.
05 Príprava 2
06 Koniec
07 Potvrdenie 1
```

Pribudla microtask potvrd() (cislo = 2) už čakalo, preto zostane pred vystavDoklad() (cislo = 1).

Objednávky: task a microtask

Druhé potvrdenie. potvrd() (cislo = 2) vypíše Potvrdenie 2 a pridá ďalší doklad na koniec frontu.

Call stack

Vrch zásobníka je hore ↑

Objednávka 2 · cislo = 2

```
function potvrd() {
  console.log(`Potvrdenie ${cislo}`);

  queueMicrotask(function
vystavDoklad() {
    console.log(`Doklad ${cislo}`);
  });
}
```

Microtask queue

Začiatok frontu je hore ↑

Objednávka 1 · cislo = 1

```
function vystavDoklad() {
  console.log(`Doklad ${cislo}`);
}
```

Objednávka 2 · cislo = 2

```
function vystavDoklad() {
  console.log(`Doklad ${cislo}`);
}
```

Task queue

Pripravené callbacky · začiatok hore ↑

Objednávka 1 · cislo = 1

```
function odovzdaj() {
  console.log(`Odovzdanie ${cislo}`);

  Promise.resolve().then(function
uzavri() {
    console.log(`Uzavretie ${cislo}`);
  });
}
```

Objednávka 2 · cislo = 2

```
function odovzdaj() {
  console.log(`Odovzdanie ${cislo}`);

  Promise.resolve().then(function
uzavri() {
    console.log(`Uzavretie ${cislo}`);
  });
}
```

Console

Poradie výpisov

```
01 Začiatok
02 Objednávka 1.
03 Príprava 1
04 Objednávka 2.
05 Príprava 2
06 Koniec
07 Potvrdenie 1
08 Potvrdenie 2
```

Microtasks pokračujú Task queue ešte nedostane priestor.

Objednávky: task a microtask

Prvý doklad. Vnorená microtask vypíše Doklad 1. Aj nová microtask predbehne ďalšiu task.

Call stack

Vrch zásobníka je hore ↑

Objednávka 1 · cislo = 1

```
function vystavDoklad() {
  console.log(`Doklad ${cislo}`);
}
```

Microtask queue

Začiatok frontu je hore ↑

Objednávka 2 · cislo = 2

```
function vystavDoklad() {
  console.log(`Doklad ${cislo}`);
}
```

Task queue

Pripravené callbacky · začiatok hore ↑

Objednávka 1 · cislo = 1

```
function odovzdaj() {
  console.log(`Odovzdanie ${cislo}`);

  Promise.resolve().then(function
uzavri() {
    console.log(`Uzavretie ${cislo}`);
  });
}
```

Objednávka 2 · cislo = 2

```
function odovzdaj() {
  console.log(`Odovzdanie ${cislo}`);

  Promise.resolve().then(function
uzavri() {
    console.log(`Uzavretie ${cislo}`);
  });
}
```

Console

Poradie výpisov

```
01 Začiatok
02 Objednávka 1.
03 Príprava 1
04 Objednávka 2.
05 Príprava 2
06 Koniec
07 Potvrdenie 1
08 Potvrdenie 2
09 Doklad 1
```

Vnorená microtask Microtask queue sa spracúva až do vyprázdnenia.

Objednávky: task a microtask

Druhý doklad. Vypíše sa Doklad 2. Po dokončení tohto callbacku môže nasledovať timer.

Call stack

Vrch zásobníka je hore ↑

Objednávka 2 · cislo = 2

```
function vystavDoklad() {
  console.log(`Doklad ${cislo}`);
}
```

Microtask queue

Začiatok frontu je hore ↑

Prázdna

Task queue

Pripravené callbacky · začiatok hore ↑

Objednávka 1 · cislo = 1

```
function odovzdaj() {
  console.log(`Odovzdanie ${cislo}`);

  Promise.resolve().then(function
uzavri() {
    console.log(`Uzavretie ${cislo}`);
  });
}
```

Objednávka 2 · cislo = 2

```
function odovzdaj() {
  console.log(`Odovzdanie ${cislo}`);

  Promise.resolve().then(function
uzavri() {
    console.log(`Uzavretie ${cislo}`);
  });
}
```

Console

Poradie výpisov

```
01 Začiatok
02 Objednávka 1.
03 Príprava 1
04 Objednávka 2.
05 Príprava 2
06 Koniec
07 Potvrdenie 1
08 Potvrdenie 2
09 Doklad 1
10 Doklad 2
```

Posledná microtask Front je prázdny; ešte sa musí dokončiť aktívny callback.

Objednávky: task a microtask

Odovzdanie prvej objednávky. Prvý timer vypíše Odovzdanie 1 a naplánuje microtask uzavri() (cislo = 1).

Call stack

Vrch zásobníka je hore ↑

Objednávka 1 · cislo = 1

```
function odovzdaj() {
  console.log(`Odovzdanie ${cislo}`);

  Promise.resolve().then(function
uzavri() {
    console.log(`Uzavretie ${cislo}`);
  });
}
```

Microtask queue

Začiatok frontu je hore ↑

Objednávka 1 · cislo = 1

```
function uzavri() {
  console.log(`Uzavretie ${cislo}`);
}
```

Task queue

Pripravené callbacky · začiatok hore ↑

Objednávka 2 · cislo = 2

```
function odovzdaj() {
  console.log(`Odovzdanie ${cislo}`);

  Promise.resolve().then(function
uzavri() {
    console.log(`Uzavretie ${cislo}`);
  });
}
```

Console

Poradie výpisov

```
01 Začiatok
02 Objednávka 1.
03 Príprava 1
04 Objednávka 2.
05 Príprava 2
06 Koniec
07 Potvrdenie 1
08 Potvrdenie 2
09 Doklad 1
10 Doklad 2
11 Odovzdanie 1
```

Vykonáva sa task Druhý timer nepreruší aktívny callback.

Objednávky: task a microtask

Uzavretie prvej objednávky. Po task opäť skontrolujeme microtask queue. Vypíše sa Uzavretie 1.

Call stack

Vrch zásobníka je hore ↑

Objednávka 1 · cislo = 1

```
function uzavri() {
  console.log(`Uzavretie ${cislo}`);
}
```

Microtask queue

Začiatok frontu je hore ↑

Prázdna

Task queue

Pripravené callbacky · začiatok hore ↑

Objednávka 2 · cislo = 2

```
function odovzdaj() {
  console.log(`Odovzdanie ${cislo}`);

  Promise.resolve().then(function
uzavri() {
    console.log(`Uzavretie ${cislo}`);
  });
}
```

Console

Poradie výpisov

```
01 Začiatok
02 Objednávka 1.
03 Príprava 1
04 Objednávka 2.
05 Príprava 2
06 Koniec
07 Potvrdenie 1
08 Potvrdenie 2
09 Doklad 1
10 Doklad 2
11 Odovzdanie 1
12 Uzavretie 1
```

Znova microtask uzavri() (cislo = 1) sa vykoná ešte pred odovzdaním druhej objednávky.

Objednávky: task a microtask

Odovzdanie druhej objednávky. Ďalší timer vypíše Odovzdanie 2 a naplánuje microtask uzavri() (cislo = 2).

Call stack

Vrch zásobníka je hore ↑

Objednávka 2 · cislo = 2

```
function odovzdaj() {
  console.log(`Odovzdanie ${cislo}`);

  Promise.resolve().then(function
uzavri() {
    console.log(`Uzavretie ${cislo}`);
  });
}
```

Microtask queue

Začiatok frontu je hore ↑

Objednávka 2 · cislo = 2

```
function uzavri() {
  console.log(`Uzavretie ${cislo}`);
}
```

Task queue

Pripravené callbacky · začiatok hore ↑

Prázdna

Console

Poradie výpisov

```
01 Začiatok
02 Objednávka 1.
03 Príprava 1
04 Objednávka 2.
05 Príprava 2
06 Koniec
07 Potvrdenie 1
08 Potvrdenie 2
09 Doklad 1
10 Doklad 2
11 Odovzdanie 1
12 Uzavretie 1
13 Odovzdanie 2
```

Ďalšia task K timeru sa dostaneme až po dokončení predchádzajúcich microtasks.

Objednávky: task a microtask

Uzavretie druhej objednávky. Po druhom timeri sa vykoná uzavri() (cislo = 2) a vypíše Uzavretie 2.

Call stack

Vrch zásobníka je hore ↑

Objednávka 2 · cislo = 2

```
function uzavri() {
  console.log(`Uzavretie ${cislo}`);
}
```

Microtask queue

Začiatok frontu je hore ↑

Prázdna

Task queue

Pripravené callbacky · začiatok hore ↑

Prázdna

Console

Poradie výpisov

```
01 Začiatok
02 Objednávka 1.
03 Príprava 1
04 Objednávka 2.
05 Príprava 2
06 Koniec
07 Potvrdenie 1
08 Potvrdenie 2
09 Doklad 1
10 Doklad 2
11 Odovzdanie 1
12 Uzavretie 1
13 Odovzdanie 2
14 Uzavretie 2
```

Znova microtask Po každej task sa microtask queue kontroluje opäť.

Objednávky: task a microtask

Všetky callbacky sa dokončili. Call stack aj oba fronty sú pre náš príklad prázdne.

Call stack

Vrch zásobníka je hore ↑

Prázdny zásobník

Microtask queue

Začiatok frontu je hore ↑

Prázdna

Task queue

Pripravené callbacky · začiatok hore ↑

Prázdna

Console

Poradie výpisov

```
01 Začiatok
02 Objednávka 1.
03 Príprava 1
04 Objednávka 2.
05 Príprava 2
06 Koniec
07 Potvrdenie 1
08 Potvrdenie 2
09 Doklad 1
10 Doklad 2
11 Odovzdanie 1
12 Uzavretie 1
13 Odovzdanie 2
14 Uzavretie 2
```

Hotovo Potvrdenia → doklady → odovzdanie 1 → uzavretie 1 → odovzdanie 2 → uzavretie 2.

Úloha: poradie výpisov

V akom poradí sa vypíšu písmená do konzoly?

```
console.log('A');

setTimeout(() => console.log('B'), 0);

new Promise(resolve => {
  console.log('G');
  resolve();
}).then(() => {
  console.log('C');
  queueMicrotask(() => console.log('D'));
});

queueMicrotask(() => console.log('E'));

console.log('F');
```

Výsledok: poradie výpisov

```
A
G
F
C
E
D
B
```

A → G → F

Synchrónny kód vrátane executora Promise.

C → E → D

Microtask C pridá D na koniec frontu, až za už čakajúcu E.

B

Callback timeru sa vykoná až po microtasks.

`async` / `await`

`async` a `await` umožňujú pracovať s Promise **postupným zápisom** bez reťazenia `.then()`.

Funkcia označená `async` vždy vracia **Promise**. `await` v nej počká na výsledok Promise a až potom pokračuje ďalším riadkom.

Počas čakania **neblokuje JS vlákno**.

DEMO

Sekvenčné a súbežné vykonávanie

Pri **sekvenčnom vykonávaní** začneme ďalší krok až po dokončení predchádzajúceho. Použijeme ho, keď druhý krok potrebuje výsledok prvého.

Pri **súbežnom vykonávaní** spustíme viac nezávislých asynchrónnych činností bez čakania na dokončenie prvej. Ich **časy čakania sa môžu prekrývať**.

Súbežnosť sama osebe **neznamená paralelné vykonávanie JavaScriptového kódu**.

Príprava dvoch objednávok

Prvú objednávku simulujeme čakaním **900 ms**, druhú **600 ms**. Použijeme funkciu `wait` z predchádzajúceho príkladu.

```
async function priprav(cislo, ms) {
  console.log(`Príprava ${cislo}`);
  await wait(ms);
  console.log(`Odovzdanie ${cislo}`);
  return `Objednávka ${cislo}`;
}
```

Sekvenčné vykonávanie

```
async function pripravPostupne() {
  await priprav(1, 900);
  await priprav(2, 600);
  console.log('Všetko hotové');
}
```

Najprv sa dokončí **prvá objednávka** a až potom začneme pripravovať **druhú**.

Celkové čakanie: **900 ms + 600 ms ≈ 1500 ms**.

Súbežné vykonávanie cez `Promise.all`

```
async function pripravSubezne() {
  const prva = priprav(1, 900);
  const druha = priprav(2, 600);

  const results = await Promise.all([prva, druha]);
  console.log(`Výsledky: ${results.join(', ')}`);
}
```

Obe prípravy spustíme **pred čakaním na výsledky**. Druhá objednávka sa dokončí skôr.

Na obe spolu čakáme približne **900 ms** – ich časy čakania sa prekrývajú.

Výsledky `Promise.all()`

`Promise.all()` vráti Promise, ktorá sa úspešne splní, keď sa úspešne dokončia **všetky vstupné Promise**.

Pole výsledkov zachová **poradie vstupného poľa**, preto je vo výsledkoch objednávka 1 pred objednávkou 2, hoci sa dokončila neskôr.

`Promise.all()` sa odmietne pri prvom spracovanom odmietnutí, ale **ostatné rozbehnuté prípravy nezruší**.

Ak potrebujeme výsledok každej objednávky aj pri čiastočnom zlyhaní, existuje `Promise.allSettled()`. Vráti pre každú vstupnú Promise záznam `fulfilled` s hodnotou alebo `rejected` s dôvodom.

DEMO

HTTP a `fetch`

HTTP je protokol, pomocou ktorého si klient a server vymieňajú dáta. Naša aplikácia je klient: pošle serveru požiadavku, napríklad na zoznam produktov, a server jej vráti odpoveď. API určuje, na aké adresy môžeme požiadavky posielať, aké údaje očakáva a čo nám vráti.

`fetch()` je funkcia na odoslanie HTTP požiadavky z JavaScriptu. Vracia Promise, pretože odpoveď zo siete nepríde okamžite. Počas čakania môže aplikácia ďalej reagovať. Odpoveď spracujeme pomocou `.then()` alebo `await`.

Výsledkom `await fetch(url)` je objekt `Response`, ktorý obsahuje napríklad HTTP status a umožňuje prečítať telo odpovede. Ak server posiela JSON, dáta získame ďalším asynchrónnym krokom: `await response.json()`.

DEMO

HTTP chyba verzus sieťová chyba

`fetch()` vráti Promise, ktorý sa **neodmietne len preto, že server vrátil chybový HTTP status**, napríklad 403, 404 alebo 500. Preto kontrolujeme `response.ok`, ktoré je `true` pri statuse **200 až 299**.

Pri **sieťovom zlyhaní** nemusíme dostať žiadny objekt `Response`. Promise z `fetch` sa odmietne a riadenie prejde do `catch`.

Samostatne môže zlyhať aj **parsovanie nesprávneho JSON tela**.

Promise v React Native komponente

Prečo nie `useEffect(async () => …)`?

Callback `useEffect` má vrátiť **cleanup funkciu alebo nič**. Async funkcia vracia Promise, preto do efektu vložíme **vnútornú async funkciu** a spustíme ju.

Cleanup a zrušenie requestu

Cleanup sa vykoná **pred ďalším spustením efektu pri zmene závislostí** a pri **unmountnutí komponentu**.

`controller.abort()` požiada o zrušenie requestu. Jeho Promise sa môže odmietnuť.

Už vykonanú akciu na serveri tým nevrátime späť.

Neaktuálne výsledky a Strict Mode

`ignore` zabráni neaktuálnej operácii zapísať **dáta, chybu aj loading** do state.

Race condition: starší request môže skončiť po novšom a prepísať jeho výsledok. Poradie dokončenia nemusí zodpovedať poradiu spustenia.

Strict Mode vo vývoji môže pridať cyklus `setup → cleanup → setup`. Efekt musí správne fungovať aj pri tomto overení.